

Chapter - 6 (Singly linked list)

Vidya Lekhan

6.1 Dynamic Memory allocation, Memory Representation of linked list, Availability stack, Types of LL - Singly, circular, doubly (only intro)

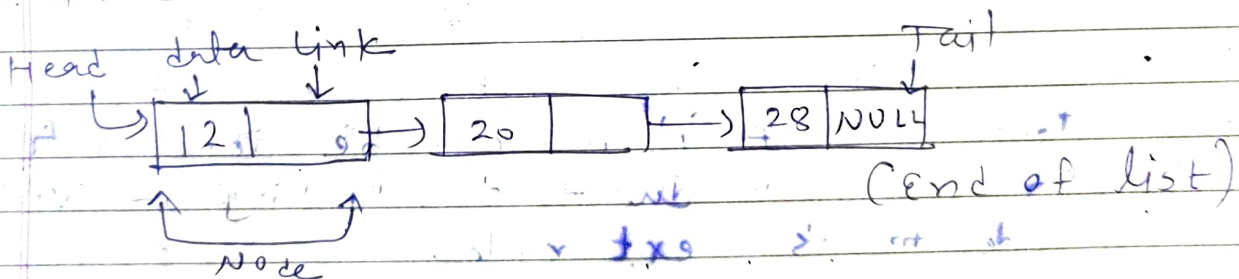
6.2 operation on Singly LL -

- ↳ insert (at first, at last, before particular value)
- ↳ Delete (at first, at last, particular value)
- ↳ (Traverse, display) (only prog. with little ^{more} ~~best~~)

6.3 implementation of Stack (push, pop, display)
implementation of Queue (enqueue, dequeue, display)

Linked List

A linked list is a linear data structure of nodes where each node contains **data** and **link** where link is a pointer that points immediate next node in the list



- Each element in Singly LL is called Node.
- Each node in list can be accessed linearly by traversing the list from Head to Tail.
- The list is linear but not contiguous the node can reside anywhere in the memory and link together to make a list.

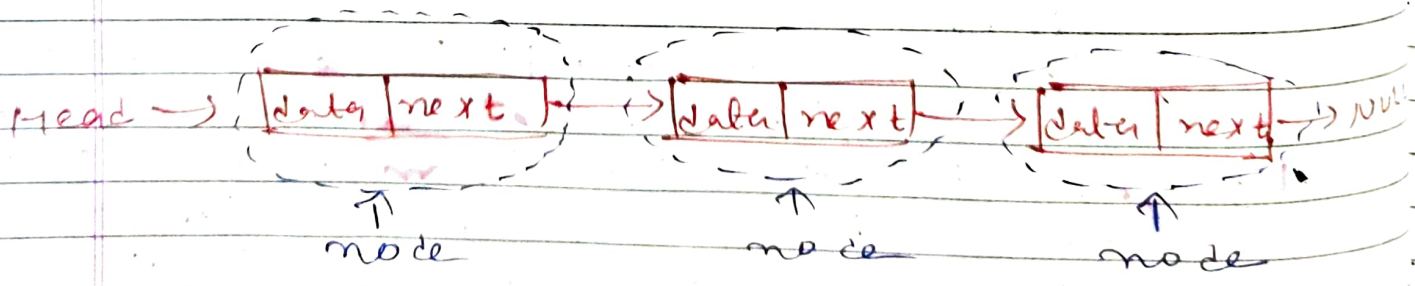
⇒ Why Linkedlist DS needed?

1. Dynamic Data structure:- Size of memory can be allocated or de-allocated at run time.
2. Ease of insertion / Deletion:- Simpler than array since no element need to be shifted after operations.
3. Efficient Memory Utilization:- as we know Linkedlist is Dynamic; the size increases or decreases as per requirement.
4. Implementation:- various DS like stack, queue, graph etc. implemented using LL.

Types of Linkedlist

1. Single-linked list:-

Traversal of items can be done in forward direction only due to the linking of every node to its next node.

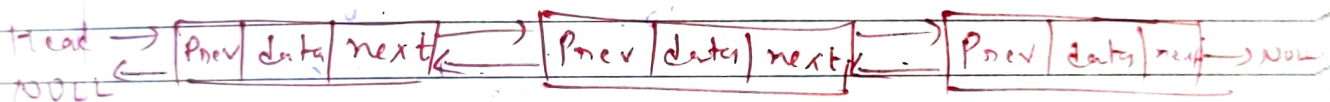


operations on Singly linked list:-

1. insertion :- it can be performed in three ways (at first, at last, before particular value)
2. Deletion :- it can be performed in three ways (at first, at last, a specific node)
3. Search :- retrieving specific node
4. Display :- displays the element of LL.

2. Doubly Linked List :-

Traversal of items can be done in both forward & backward directions as every node contains an additional prev pointer that points to previous node.



3. Circular linked List :-

It is a type of LL in which the first and the last nodes are also connected to each other to form a circle. there is No NULL at the end.



Advantages :-

- Dynamic in size, memory efficient, ease of insertion/deletion

disadvantages

- Memory usage - Extra memory usage of pointer is needed.
- Accessing a node - Random access is not possible due to dynamic allocation as node traversed sequentially from head to tail.
- Searching of element is costly.
- Traversing is costly and Reverse traversing is not possible.

→ Linked List Vs. ArrayArray

- arrays are stored in contiguous locations.
- fixed in size.
- Memory allocation at compile time.
- uses less memory.
- accessing element is easy.
- insertion/deletion is time consuming.

0	1	2	3	4
10	20	30	40	50

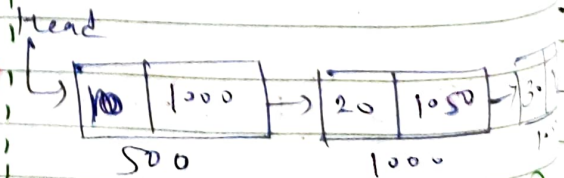
array length = 5

first index = 0

last index = 4

Linked List

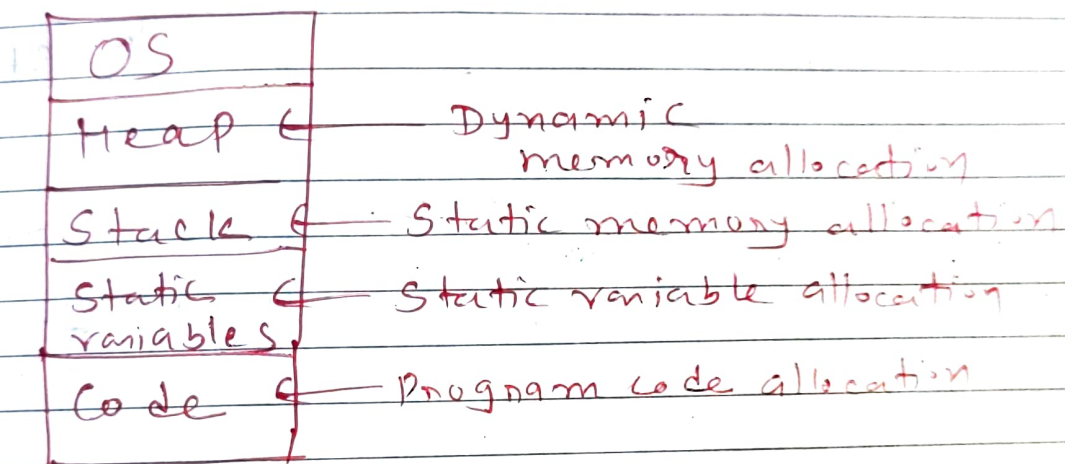
- LL are not stored in contiguous location.
- Dynamic in size.
- Memory allocation at run time.
- Uses more memory.
- accessing element is time consuming.
- insertion/deletion is faster.



→ Dynamic Memory allocation

- Dynamic memory allocation means that the memory is allocated to Java objects during run time or execution time.
- Dynamic Memory allocation takes place in Heap space, where new objects are always created and their references are stored in stack memory.

memory layout to understand difference between SMA & DMA



eg

```
p.s.v.m ( )
{
```

```
int x, y;
```

```
x = 10;
```

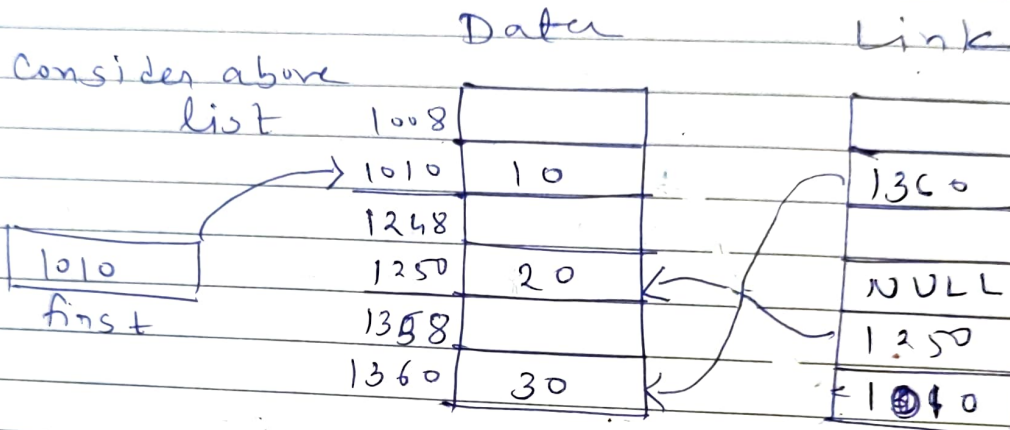
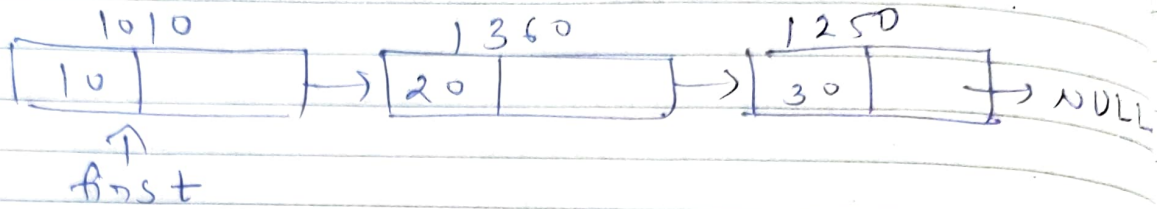
```
y = sc.nextInt();
```

```
S.o.p (x + " " + y);
```

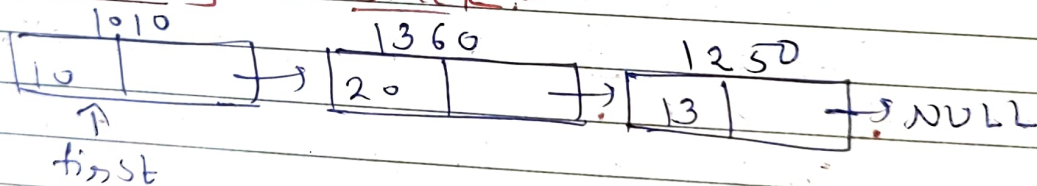
```
}
```

```
Stack x = 10
Heap y at run time
Code
p.s.v.m
{ S.o.p() }
```

Linked List Representation



Availability of Stack:-



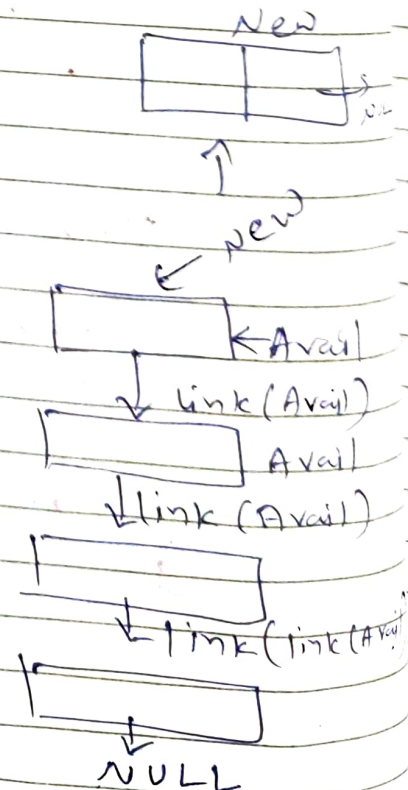
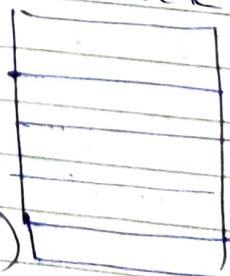
Who points to whom

$A \leftarrow B$
(A points to B)

Steps:-

1. $NEW \leftarrow AVAIL$
2. $AVAIL \leftarrow \text{Link}(AVAIL)$

Availability Stack



Steps to Create and display Singly LL

1. Create a class Node which has two attributes: data and Next.
2. Create another class, which has two attributes:- Head and Tail.
3. addNode() will add new node to the list.
 - ↳ Create a new node
 - ↳ it first checks, whether the head is equal to null which means list is empty.
 - ↳ if list is empty both head and tail will point to newly added node.
 - ↳ if list is not empty then new node will be added to end of list such that tail's next will point to newly added node and this new node becomes tail.
4. Display() - will display the list.
 - ↳ define node current which initially points to head.
 - ↳ traverse through the list till current points to null.
 - ↳ display each node and make current to next node pointer.

Creating Node :-

```
class LinkedList {  
    Node head = null; // head of list
```

```
    // Node class
```

```
    class Node {  
        int data;  
        Node next;
```

```
    // constructor to create new node
```

```
        Node (int d)
```

```
        {
```

```
            data = d;
```

```
            next = null;
```

```
        }
```

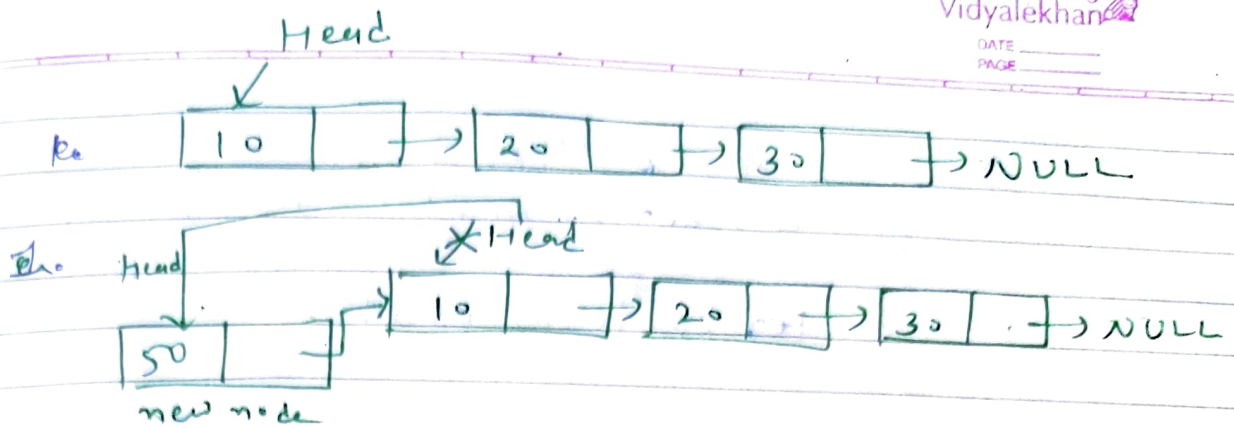
```
    }
```

```
}
```

Here I want to add multiple nodes in single LL so we need to use nested class, if we did not use nested class then every node is working independently in that case we can not generate linked list.

Inserting a node at front / first / Beginning

1. Make the first node of linked list to the new node.
2. Remove head from original first node.
3. Make the new node as head of LL.



```

void addfirst (int data)
{
    // 1. & 2. Step
    Node new-node = new Node(new data);
    if (Head == null)
    {
        Head = new-node new-node;
    }
    else
    {
        // 3. step
        new-node.next = Head;
        // 4. step
        Head = new-node;
    }
}
  
```

→ Inserting a node at last

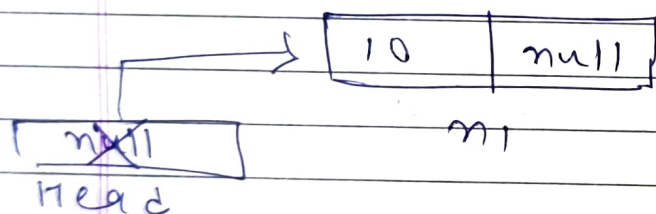
```

void addlast (int data)
{
    Node n1 = new Node (data);
    if (Head == null)
    {
        Head = n1;
    }
    else
  
```

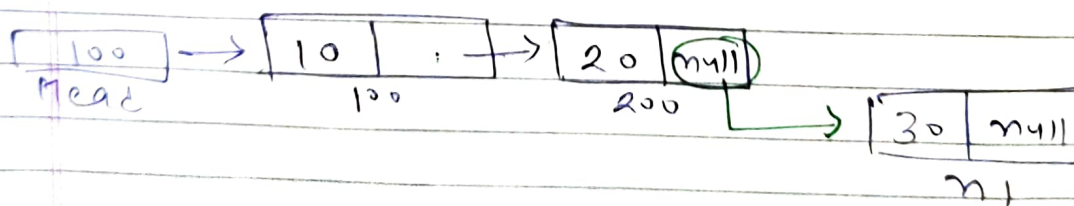
```
{ Node temp = Head  
  while (temp.next != null)  
  {  
    temp = temp.next;  
  }  
  temp.next = n1;  
}
```

Explanation :-

if block :- if we add first node means linked list is just created in that case Head will be null. So we have to set Head = n1.



In Else block, we add node at last position of Linked List



first of all we have to take one temp node, which is used to traverse at the end of the list.

temp = 100
temp.next (200) != null ✓

temp = 200

temp.next (null) = null X

out of loop

temp.next = n1

null or address n1 of address store

eg.

→ Inserting a node before particular value

```
void insertBefore(int data, int value)
{
```

```
    int flag = 0;
```

```
    Node dummy = Head;
```

```
    while (dummy != null)
```

Traverse the list and check if the value is present in list or not.

```
    {
```

```
        if (dummy.data == value)
```

```
        {
```

```
            flag = 1;
```

```
        }
```

```
        dummy = dummy.next;
```

```
    }
```

```
    if (flag == 0)
```

```
    {
```

```
        S.o.p("The value is not present in list");
```

```
    }
```

```
    else
```

```
    {
```

```
        Node n = new Node(data);
```

```
        if (Head.data == value)
```

```
        {
```

```
            addfirst(data);
```

```
        }
```

Value Present at first node

value is present in list so need to insert.

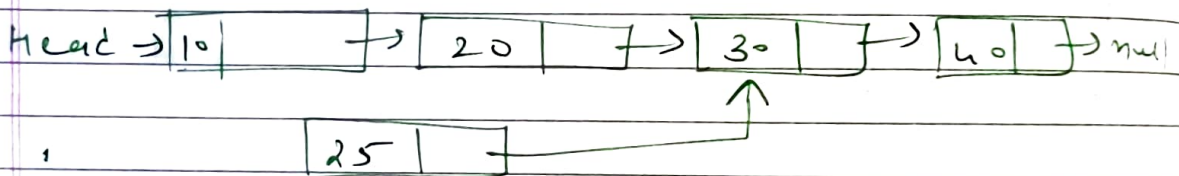
```

else
{
    Value present at other than first node {
        node temp = head;
        while (temp.next.data != value)
        {
            temp = temp.next;
        }
        n.next = temp.next;
        temp.next = n;
    }
}

```

Explanation:-

if the list is like below



If i want to add 25 before value=30 then we need to traverse our temp and stop it before 30, here we need to take our temp to 20 and then

temp.data = 20

temp.next points to 30 so we have to give that pointer to new node

So n.next = temp.next and Now break the link between 20 & 30 by updating link of 20 to 25

for that, we set the link as below.

temp.next = n |

⇒ Insert after particular value

Void insertAfter (int data, int value)

```
{
    int flag = 0;
    Node dummy = Head;
    while (dummy != null)
    {
        if (dummy.data == value)
        {
            flag = 1;
        }
    }
```

```
    dummy = dummy.next;
```

```
    if (flag == 0)
    {
```

S.O.P ("value not present in list so insertion not possible");

```
    }
    else
```

```
    {
        Node n = new Node (data);
```

```
        Node temp = Head;
```

```
        while (temp.data != value)
        {
```

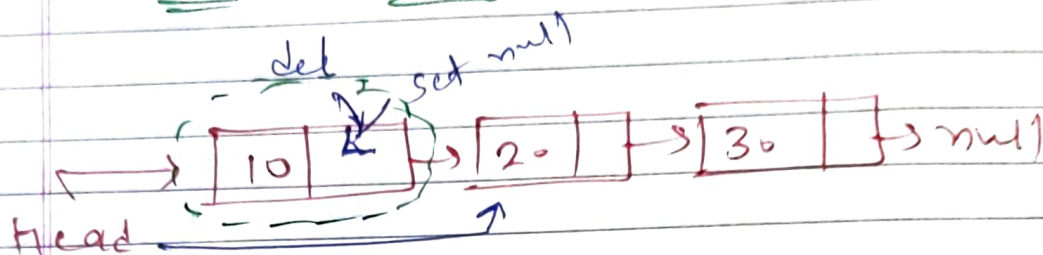
```
            temp = temp.next;
```

```
        }
        n.next = temp.next;
```

```
        temp.next = n;
    }
```

⇒ Deleting a node:-

★ → Delete at first



If I want to delete first node, who holds 10 data then, need to update next pointer of Head to data = 20 and old node's next set to null.

For this we need to follow following steps:

→ find previous node of the node, which you want to delete. (use temp.next)

→ change the next of that node

→ free the memory of the node, you want to delete.

⇒ void delFirst ()

{

if (Head == null)

{

S.o.p ("underflow");

}

else

{

Node del = Head;

Head = Head.next;

del.next = null;

}

⇒ Delete at last:-

→ Traverse the list and stops at the second last position and then make it's next as null

```
void dellast()
```

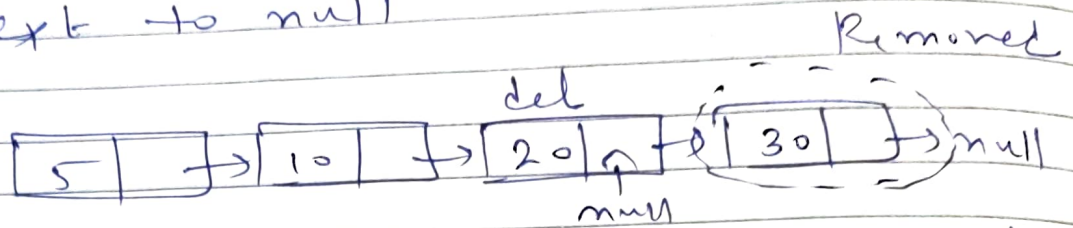
```
{
    if (Head == null)
    {
        s.o.p ("underflow");
    }
    else if (Head.next == null)
    {
        Head = null;
    }
    else
    {
        Node del = Head;
        while (del.next.next != null)
        {
            del = del.next;
        }
        del.next = null;
    }
}
```

explanation:-



del.next = 5's next = 10
del.next.next = 10's next which points 20

→ traverse till the node hold 20 as data and then make that node's next to null



del.next = 20's next = null

⇒ Delete particular value

```
void delvalue(int value)
{
```

```
    int flag = 0;
```

```
    Node dummy = Head;
```

```
    while (dummy != null)
```

```
    {
```

```
        if (dummy.data == value)
```

```
        {
```

```
            flag = 1;
```

```
        }
```

```
        dummy = dummy.next;
```

```
    }
```

```
    if (flag == 0)
```

```
    {
```

```
        S.o.p("value not found in list so  
        deletion not possible");
```

```
    }
```

```
    else
```

```
    {
```

```
        if (Head.data == value)
```

```
        {
```

```
Node del = Head;
Head = Head.next;
del.next = null;
```

```
}
else
```

```
{
```

```
Node temp = Head;
while (temp.next.data != value)
{
    temp = temp.next;
```

```
}
Node q = temp.next;
temp.next = q.next;
q.next = null;
```

```
}
}
```

explanation:

Head → [10] → [20] → [30] → [40] → null

i want to delete the node hold's value = 30

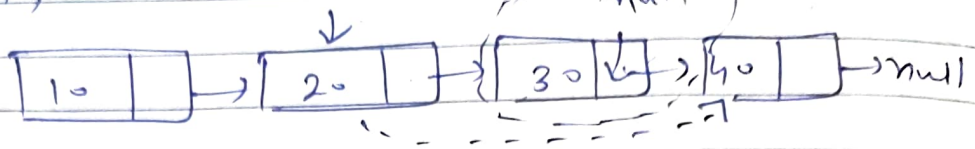
→ first of all we check that value exist in the list or not.

→ then checks the value, want to delete is first node or not if it is first node then

Head → [10] → [20] → [30] → [40] → null

need to update head to head.next and that node's next to null.

→ if the value is not at first place then,



if i want to delete 30 then need to stop at 20's node, then set the next link of 20 connect with 40, and set next of 30 to null

So, here in code i traverse temp to 20's location and also create one reference to 20's node

temp = 20

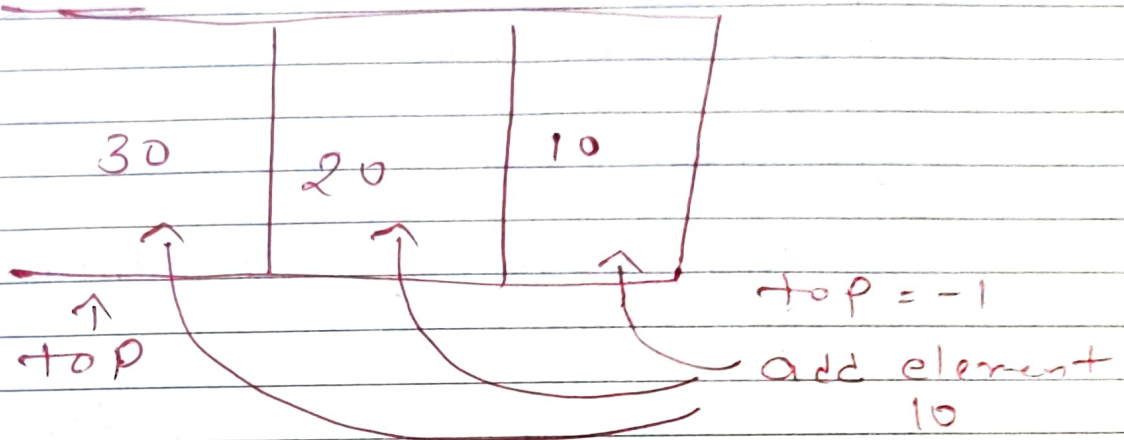
q = 20's next = 30's node

free node holds and 30's set-link of 20

next of 30 is 40 so set temp.next = q.next = 40's node and q.next = null

Implement stack using linked list:-

Assume the stack's representation is as below



then add 20 using add first
then again add 30 using add first
→ now if i want to pop all element
it will popped from top so list will
give output like "30 -- 20 -- 10 -- null"

So need to use del first method to
popped out the elements.

For implementation of Stack, need to
use add first() & del first() method of
linked list.

Prog :-

```
class SLL
{
    class Node {
        int data;
        Node next;
```

```

Node (int d)
{
    data = d;
    next = null;
}
Node top = null;

```

```

void push (int data)
{
    Node n = new Node (data);
    if (n == null)
    {
        S.o.p ("Stack overflow");
    }
    else
    {
        if (top == null)
        {
            top = n;
        }
        else
        {
            n.next = top;
            top = n;
        }
        S.o.p ("element inserted successfully");
    }
}

```

```

void pop ()
{
    if (top == null)
    {
        S.o.p ("underflow");
    }
}

```

```
else  
{
```

```
    Node del = top;  
    top = top.next;  
    del.next = null;
```

```
}
```

```
}
```

```
void display()
```

```
{
```

```
    Node temp = top;  
    if (temp == null)
```

```
{
```

```
        S.o.p ("Stack is empty");
```

```
}
```

```
    while (temp != null)
```

```
{
```

```
        S.o.p^int (temp.data + " -> ");
```

```
        temp = temp.next;
```

```
}
```

```
    S.o.p.println();
```

```
}
```

```
}
```

```
class Main {
```

```
    p.s.v.m ()
```

```
{
```

```
        SLL s = new SLL();
```

```
        s.push(10);
```

```
        s.push(20);
```

```
        s.push(30);
```

```
        s.display();
```

```
        s.pop();
```

```
        s.display();
```

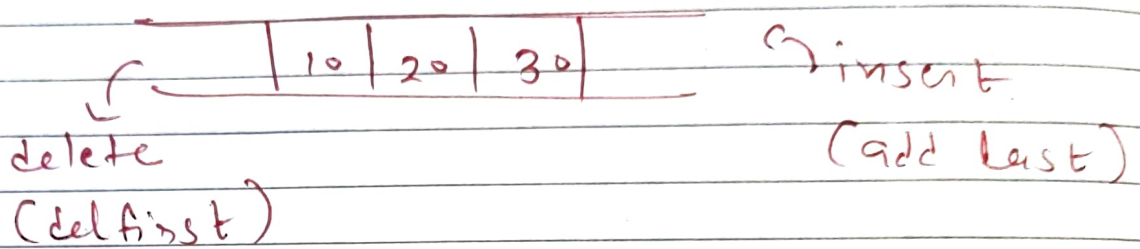
```
}
```

```
}
```

Implement Queue using Linked List

To implement Queue, we need to apply addlast and delfirst method on linked list.

Representation of Queue using linked list is as below.



Program:

```
class SLL
```

```
{
```

```
    class Node {
```

```
        int data;
```

```
        Node next;
```

```
        Node (int d)
```

```
        {
```

```
            data = d;
```

```
            next = null;
```

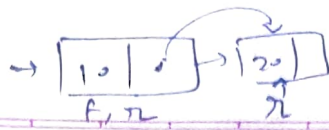
```
        }
```

```
    }
```

```
    Node front, rear = null;
```

```
    void enqueue (int data)
```

```
    {
```



```

Node n = new Node(data);
if (n == null)
{
    s.o.p("overflow");
}
else
{
    if (front == null)
    {
        front = n;
        rear = n;
    }
    else
    {
        rear.next = n;
        rear = n;
    }
}

```

```

void DeQueue()
{
    if (front == null)
    {
        s.o.p("underflow");
    }
    else
    {
        s.o.p("front.data is deleted");
        front = front.next;
    }
}

```

```

void getfront() {
    if (front == null)
    {
        s.o.p("Queue is empty so no front is there");
    }
}

```

else
 { s.o.p ("front element:" + front.data)
 }
 }

void get rear ()

```

{
    if (front == null)
    {
        s.o.p ("Queue empty so no rear is there")
    }
    else
    {
        s.o.p ("rear element:" + rear.data)
    }
}

```

void display ()

```

{
    Node temp = front;
    while (temp != null)
    {
        s.o.p (temp.data + "--> ");
        temp = temp.next;
    }
    s.o.p ("null");
}

```

class main {

p.s.v.m ()

{

SLL s = new SLL();

s.enqueue(10); s.enqueue(20);

s.enqueue(30); s.display();

s.dequeue();

}

}